

Dispute Autopilot

An AI agent that wins back money for online sellers - safely.

The decision

Build **one** product, end to end: an agent that handles PayPal disputes for a seller. A buyer opens a dispute, the agent gathers evidence, drafts the response, the seller approves with one click, PayPal receives it, and a webhook confirms the result. This is the strongest version of your idea because it hits a real, expensive pain for business users, it needs real PayPal APIs (not a simulated checkout), and the whole story fits in a 3-minute video.

Why not FlowPay / Circles? The group-buying idea ends with a "purchase" that the PayPal sandbox cannot really complete, so the climax would be faked. Disputes have a fully real loop inside the sandbox. We keep the best ideas from FlowPay (approval gate, policy rules, idempotency, audit trail) and drop the rest.

How it scores against the judging criteria

Criterion (equal weight)	What the judge must see
Technological Implementation	Real Disputes API calls, verified webhooks, idempotent processing, structured LLM output, a deterministic approval gate.
Design	One clean product: a dispute queue, a case page, a single Approve button. No chat window taking over the screen.
Potential Impact	A number: minutes per dispute, manual vs agent (measured in the demo), and dollars of disputed funds handled before the deadline.
Innovation / Idea	The agent <i>acts</i> on a back-office problem instead of recommending products. Evidence-first, deadline-aware triage.
Presentation	One story in under 3 minutes: dispute arrives, evidence assembled, approved, resolved. Nothing else.

Honesty rule: the sandbox has no real win rates. Never show a win percentage you did not measure. Show time saved and amounts handled, and label any projection as a projection. Judges and Devpost can disqualify misleading claims, and it costs you trust.

New in this version: five stand-out features (section 2), a team plan with roles and working rules (section 4), a week-by-week schedule with owners (section 5), an updated video script, and a legal check for team submissions.

1. Scope: what ships and what does not

Everything below is sorted by one question: does it appear in the 3-minute video? If not, it is cut or postponed.

MUST HAVE (the demo)	SHOULD HAVE (if time)	CUT (do not build)
• Webhooks: signature check + dedupe • Evidence Vault (capture at payment) • Cited drafts + claim verifier • Triage + Evidence Strength Score • Policy checks + approval screen • Submit evidence through the API • Resolved-webhook status update • AG Grid dispute queue • Trust Lab (6 attacks) • Dispute Bench (40 scenarios) • Audit log per dispute	• "Money recovered" counter • Failure path (PayPal rejects) • Postman collection of the flows • Natural-language grid filter • Parked: seller-set auto-submit rules (only if all else is done by Oct 30)	• Voice • Group payments / Circles • Product search • A second backend service • WebSockets (use polling) • Redis, Kafka, Kubernetes • Multi-user roles and teams • Fake win-rate analytics

What we keep from your FlowPay prompt

- AI proposes, backend validates, human approves, PayPal executes, webhook confirms, database records.
- Money stored as integer minor units (cents), never floating point.
- Idempotency on every state-changing PayPal call; never retry a financial POST blindly.
- Buyer messages and product text are untrusted data and can never become instructions to the model.
- Every action writes an audit row. The Activity timeline is part of the demo.

Stack (keep it to one service)

Layer	Choice	Reason
App + API	One backend in the language you ship fastest (TypeScript/Next.js or Go)	A second service doubles setup and failure points. Do not split into Go + Python.
Database	PostgreSQL	Strong constraints for status and money; easy hosting.
AI	Any LLM with JSON-schema output	Hackathon allows any AI tool. Validate output against a schema before use.
Grid	AG Grid (dispute queue and case table)	Real use: sort, filter, deadline column, row detail. Targets the AG Grid prize.
Hosting	Render (web service + Postgres)	Gives judges a live URL and targets the Render prize.
Payments	PayPal sandbox: Disputes API, webhooks, OAuth	PayPal must be central to the project.

2. Five things that make it stand out

A good idea is the minimum. These five are chosen because a judge can **see, test, or measure** each one in the video, and because most entrants will not bother. They add roughly 8 to 10 person-days, which is why the plan is now for a team.

1. Evidence Vault: win it before it is filed | Innovation, Impact

What	The demo storefront takes real sandbox checkouts. When payment is captured, a webhook snapshots the order, items, address and timestamps into a vault, and later adds tracking and delivery proof. When a dispute arrives, the evidence packet already exists.
How	Capture webhook writes vault records; each item gets a SHA-256 hash and timestamp so tampering is visible. The evidence builder reads only from the vault. Tracking is also sent to PayPal's tracking endpoints (confirm in current docs).
Proof	In the video a dispute arrives and the packet is already complete: "ready in 4 seconds."
Honest limit	Carrier events come from a simulator and are labelled DEMO DATA. A real merchant would connect a real carrier.

2. Cited drafts, verified by code | Technological Implementation

What	Every factual sentence in the drafted response is linked to an evidence item. A verifier written in plain code checks each date, amount, tracking number and address against the vault. Anything unverified is flagged and blocks approval.
How	The model returns JSON: sentence, claim type, evidence ids. Verifier code (not the model) compares values. The UI shows citation chips, and the Approve button stays disabled while any claim is unverified.
Proof	Show a deliberately wrong draft being blocked: "tracking number does not match the vault."
Honest limit	It proves facts are accurate, not that the argument will persuade PayPal.

3. Money-and-clock triage | Impact, Design, AG Grid prize

What	The queue is sorted by money at risk and time left. A deterministic Evidence Strength Score (0 to 100) measures how much of the evidence PayPal expects for that reason you actually hold. A seller-set rule turns it into FIGHT, ACCEPT, or ASK A HUMAN. A banner shows "\$1,240 at risk, 2 expire in 48 hours."
How	Rules live in config and run as pure functions with unit tests. AG Grid columns: at risk, time left, strength, recommendation. Status uses text labels as well as color.
Proof	The sort order changes as deadlines approach (use test deadlines in the demo).
Honest limit	The score is evidence coverage, not a win probability . Never show it as a chance of winning.

4. Trust Lab: live attack suite | Technological Implementation, Presentation

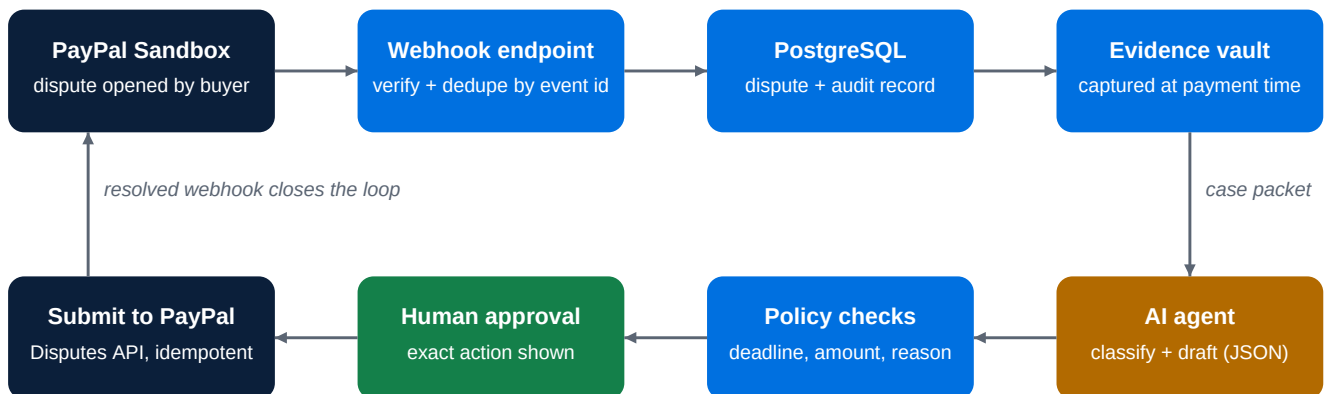
What	A page with a Run attacks button that tries six attacks on your own system: forged webhook signature, replayed event, buyer message with an injected instruction, tampered amount in an approval, approving an expired dispute, and a double-click double submit. Each row shows BLOCKED and the audit entry that proves why.
How	A test harness calls the same endpoints a real client would. Results are stored, and the same suite runs in CI on every merge.
Proof	Twenty seconds in the video: six BLOCKED rows.
Honest limit	It tests what you built. Do not claim the system is unhackable.

5. Dispute Bench: a published benchmark | Technological Implementation, Impact, credibility

What	Forty scripted dispute scenarios (different reasons, missing evidence, hostile buyer messages, near-deadline cases). One command, make bench, prints: evidence-complete rate, claims-verified rate, unsafe actions executed (must be 0), and median time against a hand-timed baseline.
How	Scenarios are JSON fixtures. Ten run live on the sandbox; the other thirty run the same pipeline against a stubbed PayPal client, and the results table says which is which. A teammate times 5 disputes by hand for the baseline.
Proof	One frame with the numbers and a link to the results file in the repo, dated, with the model name.
Honest limit	These are test scenarios, not real-world win rates. Say so on the slide.

Considered and parked: seller-set auto-submit rules (the seller approves a rule once, not every action) and a prevention-insights panel. Both are good, but only add them if everything above is finished and tested by Oct 30.

3. Architecture



Seven rules that make this credible to judges

Rule	How it is enforced
The AI never submits anything	The model returns a JSON draft only. Code validates it, policy checks it, and the backend sends it after a human approves.
Webhooks are the truth	A dispute is marked resolved only after a verified webhook (use PayPal's webhook signature verification). The UI never decides outcomes.
Duplicates do nothing	Store each PayPal event id with a unique constraint. A replayed event returns 200 and changes nothing.
Untrusted text stays data	Buyer messages go into the prompt inside clearly marked data blocks. Tool arguments are re-validated; a message saying "refund everything" has no power.
No claim without a source	Each sentence in a draft cites vault evidence. Code verifies the values; unverified claims block approval.
Secrets stay on the server	Client id and secret in environment variables only. Commit .env.example with blank values. Run a secret scan before every push.
Everything is audited	Every event, AI draft, approval, and API call writes a row with timestamp, actor, and dispute id.

Data model (9 tables, nothing extra)

Table	Purpose
disputes	PayPal dispute id, transaction id, reason, amount (cents), currency, status, response deadline, outcome
paypal_events	event id (unique), type, payload, processed_at
demo_orders	Orders placed through the demo storefront with real sandbox checkout; products and carrier events are labelled DEMO DATA
evidence_items	dispute id, type, source, content, SHA-256 hash, captured_at (the vault)
ai_drafts	dispute id, classification, draft text, schema-valid flag, model name
approvals	draft id, approver, exact action snapshot, decision, timestamp
audit_log	dispute id, actor (system / ai / user / paypal), action, detail, timestamp
bench_runs	scenario id, live or stubbed, result, model name, run date
attack_runs	attack name, outcome, audit reference, run date

Data honesty: PayPal disputes and transactions come from the sandbox. Order, tracking, and chat records come from a small demo store you seed. Label them DEMO DATA in the UI and README. Real sandbox calls plus clearly labelled demo inputs is fully legitimate. Faking PayPal responses is not.

4. Working as a team

A team wins by splitting the work cleanly and merging often, not by everyone touching everything. If you are fewer than four people, merge role C into A or D.

Role	Owns	Main deliverables
A. Tech lead + PayPal	Disputes API, OAuth, webhooks, idempotency, policy checks, Render deploy, merges to main	Spike by Oct 10, verified webhooks, vault capture, submit-on-approve, failure path
B. Agent + verification	Classifier, drafter, citation verifier, Evidence Strength Score, Dispute Bench	Schema-valid drafts, claim verifier, triage rules, bench results
C. Product + frontend	Design system, AG Grid queue, case page, approval card, Trust Lab screen	Screens with loading, error and empty states; mobile; accessibility
D. Quality + story	Demo storefront and carrier simulator, CI, attack suite, README, video, Devpost form	Clean-machine setup test, 6 attacks, storyboard, final submission

Eight working rules

- **Contracts first.** By Oct 9, merge the JSON schemas (dispute, evidence item, draft with citations, action) and one API spec file. Everyone builds against it, with mocks, so work runs in parallel.
- **One owner per folder.** Changes there need the owner's review. Pull requests stay small (about 300 lines) and merge daily.
- **Main is always deployable.** CI must pass (lint, tests, secret scan) before merge. Unfinished work hides behind a feature flag.
- **15-minute daily sync** plus one written status thread. A blocker older than 4 hours goes to the tech lead.
- **Integration Friday.** Every Friday, someone who did not build it runs the full flow on the hosted URL and logs bugs.
- **Secrets.** Share sandbox credentials through a password manager, never chat or Git. Each developer uses their own sandbox app locally; the demo uses one.
- **Definition of done.** Works on the hosted URL, has a test, writes an audit row, and has one line in the README.
- **Decisions.** The tech lead settles technical disputes in 10 minutes; one person owns scope. The Nov 3 freeze is not negotiable.

Team legal check (do this before submitting). The rules require that the submission is solely owned by the Entrant, with no other person holding rights in it. Enter as one Organization (for example your registered company) or as a Team with one appointed Representative, and have every contributor sign a short written contribution/IP agreement. Agree in writing how a prize would be shared: the Representative allocates it, and winners must pass identity verification and tax forms. Also confirm nobody on the team works for PayPal or Devpost or is a close relative of someone who does, and that your country is not on the prohibited list in the current rules. I am not a lawyer; have someone check the agreement.

5. Step-by-step plan: 5 weeks, one checkpoint each

Each week ends with a test you can run. If a checkpoint fails, fix it before starting the next week. Feature freeze is Nov 3. If you fall behind, cut in this order: bench scenarios (40 to 20), then attacks (6 to 4), then triage rules. Never cut the vault or the claim verifier; they are the core.

Week	Who does what	Done when
Week 1 Oct 6 - 12 Prove it + contracts	A: sandbox seller + buyer accounts and REST app; open a dispute, fetch it, receive its webhook, submit evidence. Public repo, MIT license, hello-world on Render. B: JSON schemas incl. cited-draft format; LLM test harness; first 10 bench scenarios. C: design system, wireframes (queue, case, approval); AG Grid skeleton on mock data. D: demo storefront skeleton with real sandbox checkout; CI with secret scan; stubs for the 6 attacks.	A real sandbox dispute goes from created to evidence-submitted (fallback below if not by Oct 10), and schemas and API spec are merged.
Week 2 Oct 13 - 19 Pipeline + Vault	A: webhook signature check, dedupe, dispute sync, capture webhook to vault; money in cents. B: evidence builder reads the vault; Evidence Strength Score and triage rules with unit tests. C: queue page on real data; case page v1. D: carrier simulator (labelled), tracking pushed to PayPal; audit timeline view; first 2 attacks working.	A real sandbox checkout creates a vault record within seconds, and a new dispute shows up in the queue with a strength score.
Week 3 Oct 20 - 26 Agent + proof	A: submit on approve, resolved webhook, idempotency keys, policy checks. B: classifier, cited drafter, claim verifier, prompt-injection defense. C: approval card with exact action, citation chips, blocked state, at-risk banner. D: all 6 attacks in the Trust Lab; make bench runs the 40 scenarios once.	Happy path works with zero manual database edits; a deliberately wrong draft is blocked; all 6 attacks show BLOCKED.
Week 4 Oct 27 - Nov 2 Product + numbers	A: failure path (PayPal rejects), rate limits, error handling; run 10 bench scenarios live on the sandbox. B: fix what the bench exposes; freeze bench numbers; time a teammate doing 5 disputes by hand as the baseline. C: polish, states, mobile, accessibility. D: a stranger tests the hosted URL; README draft; storyboard and a full rehearsal.	A stranger completes the flow without help; bench numbers and baseline are recorded.
Week 5 Nov 3 - 9 Freeze + ship	All: Nov 3 feature freeze, bug fixes only. D: test the README on a clean machine; collect signed contribution agreements; record 3 or 4 takes; upload to YouTube (public). A (Representative): Devpost draft by Nov 7; final submit by Nov 10.	Submitted 2 days early. Nov 11 - 12 is buffer only.

Fallback if sandbox disputes cannot be created by script: open them by hand from the sandbox buyer account's Resolution Center and document the exact steps in the README. Webhooks, evidence submission, and resolution stay real; only the buyer's click is manual.

Deadline: Nov 12, 12:00 pm Pacific, about 9:00 pm in Lagos. Do not plan to submit on the last day.

6. The 3-minute video

Write this script in Week 3 and build toward it. Put it on one page, rehearse it twice, and cut anything that does not fit.

Time	On screen	What you say
0:00 - 0:15	Dashboard banner: disputes, money at risk, time left (triage)	"Sellers lose money on disputes they cannot answer in time. Here, \$1,240 is at risk and two cases expire in 48 hours."
0:15 - 0:45	Buyer opens a sandbox dispute; webhook fires; a row appears already carrying a complete evidence packet	"The proof was captured when the buyer paid. The packet was ready in seconds, not hunted down by hand."
0:45 - 1:20	Case page: Evidence Strength Score, FIGHT recommendation, drafted response with citation chips	"Every sentence points to a piece of evidence."
1:20 - 1:40	A wrong draft is blocked: tracking number does not match the vault	"Code, not the AI, checks every fact. Unverified claims cannot be sent."
1:40 - 2:05	Approve the exact action, PayPal API result, resolved webhook, audit timeline	"Nothing is sent until the seller approves. PayPal confirms by webhook."
2:05 - 2:30	Trust Lab: run six attacks, six BLOCKED rows	"Forged webhooks, replays and injected instructions all fail, with proof in the audit log."
2:30 - 2:55	Bench results frame and repo link	"40 scenarios, zero unsafe actions, median X seconds versus Y minutes by hand. Open source, run it yourself."

Video rules from the official rules

- Under 3 minutes (judges need not watch longer), public on YouTube, link added in the form.
- Must show the project running on the device it was built for, working end to end.
- No third-party trademarks and no copyrighted music unless you have permission.
- English, or provide an English translation.
- Fill in the real bench numbers (X and Y) only after you measure them.

7. Submission checklist

Item	Requirement	Status
Project	Built with the PayPal developer platform (sandbox) plus an AI tool; PayPal is central	☐
Description	Features and functionality, plus what is new (project is built during the period)	☐
Public repo	GitHub/GitLab/Bitbucket, public, open-source license visible in the About section (MIT)	☐
Working demo	Hosted URL, or full setup and run instructions; include test accounts judges need	☐
Video	Under 3 minutes, public YouTube link	☐
Testing access	Free and unrestricted for judges until the Judging Period ends (Dec 15)	☐
Secrets	No credentials in code or Git history; .env.example only	☐
Bench results	Committed to the repo with date, model name, and scenario list; test data labelled	☐
Entrant and IP	Organization or Team with one Representative; signed contribution agreements	☐
Winner paperwork	If you win: identity verification and tax forms (non-US residents may need a W-8BEN)	☐

Prize targeting (a project can win one main prize plus one sponsor prize)

Target	Why it fits	Do not
Grand Prizes (\$12k / \$8k / \$5k)	Real end-to-end flow, measured results, visible safety design	Add features that are not in the video
Best Use of PayPal + AI, Agentic Commerce, Most Impactful (\$5k each)	Agent acts on real Disputes APIs and webhooks behind an approval gate; the vault prevents losses before they happen	Describe it as a chatbot
AG Grid (\$5k / \$2k / \$1k x3)	Queue sorted by money at risk and time left, with strength and recommendation columns	Use a plain table with a grid label
Render (credits)	Host the web service and database on Render	Add Render only for the badge

Risks and answers

Risk	Answer
Sandbox dispute behavior differs from live PayPal	Prove the flow in Week 1; state limits in the README; never claim live results.
Too many features for 5 weeks	Cut order is fixed (bench, attacks, triage). Vault and verifier are protected.
Team integration pain	Contracts first, daily merges, Integration Friday on the hosted URL.
Weak or invalid LLM output	Schema validation, code-based claim verification, human approval.
Misleading numbers	Evidence Strength Score is coverage, not a win chance. Bench uses test scenarios and says so.

First 3 actions today: (1) Role A creates the sandbox accounts, REST app, and opens one test dispute; (2) Role D creates the public repo with the MIT license and CI; (3) everyone agrees roles and the Oct 9 contracts deadline in writing. Once step 1 works, the project is real.